LIS: access control, updates and another-machine setup

Implementation handoff - 15 September 2026

1. The central-server approach

Use one dedicated, always-on hospital server for PostgreSQL, the LIS backend and the web frontend. Hospital staff use a browser and their individual LIS accounts. They do not install Node, PostgreSQL, the source project or Docker on each workstation. The server needs its runtime and the application images once; installation cannot require literally zero software or transfer.

On the same network, staff open the server address, for example http://192.168.1.50:8081. Replace the example IP with the real server IP. localhost on a staff PC points to that staff PC, not the hospital server. Reserve the server IP in DHCP or configure hospital DNS. The server must stay on; plan power backup, disk monitoring and tested backups.

Across different networks, ask hospital IT to provide an authenticated VPN/private connection and HTTPS. This has not been configured by these changes. Do not expose PostgreSQL or the backend directly to the public internet. HTTP examples below are for controlled setup testing, not a public clinical deployment. Phones and other computers need a compatible browser and permitted network access; being on Wi-Fi alone does not guarantee routing/firewall access.

The existing LIS_COMMERCIAL_DEPLOYMENT_AND_LICENSING_PLAN.pdf covers the wider commercial roadmap. This guide records the implemented access changes and practical deployment steps. A signed license, machine activation, automated installer, updater, TLS/VPN and production security hardening remain separate work. Docker image tags are version labels, not copy protection. A customer controlling the server can inspect its container contents.

2. Who may enter, authorize or remove a report?

- Technician: register patients, submit samples and use Submit & enter values; enter/save laboratory values. Cannot authorize or remove records without the additional required role.
- Report authorizer: review and authorize reports, print/download released reports. To enter values too, also assign Technician.
- Technician + Report authorizer: both capabilities, including authorization of results that the same person entered, as requested. This is not a mandatory two-person review workflow.
- Administrator: all of the above, including protected removal and full staff administration.
- Doctor: hospital-wide patient/status access, authorized results and append-only clinical notes; no laboratory value editing or authorization from the Doctor role alone.

- IT: delegated clinical-staff/department management; no clinical access from IT alone. IT cannot assign Administrator/IT or edit accounts containing either elevated role. Administrators control these elevated assignments.
- Investigation manager: create, edit, archive, restore and permanently delete unused investigations in Tests. Only an Administrator can assign this role. Investigations already used in patient orders cannot be permanently deleted and must be archived.

In Team, create or edit a staff account and select one or more Assigned roles checkboxes. Save access. Permissions combine across selected roles. Assigning Doctor plus Technician intentionally adds Technician's draft/result-entry access. The role list is not an additional deny-list. Departments identify affiliation, not patient-visibility restrictions. Existing single-role accounts remain supported.

Server-side checks reload account status and roles on every authenticated request; removing a role or disabling a user affects an existing token on its next request. The browser refreshes account state, and reloading reflects changes immediately. Staff cannot disable their own account or change their own roles. Administrator access is protected from removal of the last active administrator.

Administrator-controlled delegation is retained: an administrator can assign IT or Investigation manager. IT can manage clinical roles but cannot grant Investigation manager. There is no arbitrary per-action permission editor or separate switch requiring a second authorizer in this implementation.

3. Protected removal is archival, not physical erasure

Only an active Administrator may use Delete report. The server requires the exact accession number, a reason of 10-500 characters and the acting administrator's current password. Five wrong password confirmations block further confirmation attempts for the remainder of a 15-minute window. The counter is stored in the database, not browser memory.

Published reports are marked removed with a VOID audit action; unreleased records use ARCHIVE. The order's patient snapshot, values, notes and audit history remain stored. Removed records disappear from ordinary worklists, patient searches and report access. Passwords are not included in removal audit events. Concurrent removal attempts do not create duplicate successful removal events.

There is no separate patient-master deletion API in this LIS: patient details are snapshots attached to lab orders. Removing one order does not remove every order for that patient and does not delete an external HMS patient. Existing downloaded PDFs or printed reports cannot be recalled. There is no new restore screen or permanent-purge workflow. Retention and access policies need hospital approval before real use.

These are application controls, not a claim of complete security or regulatory certification. Someone with unrestricted database/server access can bypass application controls. Restrict that access, protect backups, secure transport and review operational security separately.

4. Apply source-code changes on the current machine

Docker does not automatically copy saved source changes into the running production containers. Building creates an image; starting/recreating the services applies it. Work in the same project folder and with the same Compose project name used by the existing installation, so its database volume is reused.

Before an upgrade, arrange a maintenance window and take a database backup. Replace lis below if POSTGRES_USER or POSTGRES_DB differ. Use a unique backup name for each run. The database name/user are not necessarily your LIS login.

```
cd D:\lis
docker compose exec -T db pg_dump -U lis -d lis -Fc -f /tmp/lis-before-access-update.dump
docker compose cp db:/tmp/lis-before-access-update.dump ./lis-before-access-update.dump
```

Confirm the command succeeded, copy the dump to protected off-server storage and test restoration in an isolated environment. Do not include real hospital backups in a buyer installation. Avoid PowerShell binary redirection for custom-format dumps.

Set a unique release version in the ROOT .env (next to docker-compose.yml), for example LIS_VERSION=1.1.0 for this version if unused. backend/.env is for non-Docker backend execution; Compose uses the root .env. Then:

```
docker compose config --quiet
docker compose up -d --build backend frontend
docker compose ps
docker compose images
docker compose logs --tail 60 backend
```

The backend startup runs the idempotent SQL migration automatically. Check that both services become healthy. Visit the configured frontend port (this local installation uses http://localhost:8081), then reload the browser and test Team, registration, entry, authorization and a synthetic removal. Do not use a real patient to test deletion. If the deployment uses another FRONTEND_PORT, use that port instead.

Do not run docker compose down -v: it removes Compose-managed database volumes. Do not delete the volume in Docker Desktop. A folder/project-name change can create a different volume and make records appear missing. Image rollback is not automatically database rollback; verify schema compatibility or restore a tested backup under a controlled recovery plan.

For a non-Docker source installation, build frontend/backend, run npm.cmd run migrate in backend with the intended database configured, and restart the actual application processes. A frontend build alone does not update a running Docker image.

5. Why old images reappear, and how versions work

backend and frontend are service names. The configured images are now akash-lis-backend:${LIS_VERSION:-1.1.0} and akash-lis-frontend:${LIS_VERSION:-1.1.0}. Both use the same release variable. If you delete an image and run Compose with build enabled, Docker creates the needed image again. That is expected.

A tag is a movable name for an image ID. Rebuilding the same tag can leave the earlier image untagged; an older container may still reference it. Prefer a new, never-reused release tag for each delivered version, keep a release record and verify the running services with docker compose images. This change does not automatically publish images to a registry or add an application version badge.

After confirming the new release and retaining your agreed rollback copy, inspect docker image ls and docker ps -a. Remove only specific obsolete, unused image tags with docker image rm <exact-old-tag>. Docker may refuse if an existing container still uses the image. Do not use forced/broad pruning to make Desktop look tidy; never delete database volumes as image cleanup. The names visible in the Containers view and Images view describe different objects.

6. Run on another machine today

For a developer/test source setup, install a compatible Docker runtime on the central machine, copy the project without .env secrets, backups, node_modules, .git or test-results, create a new root .env from .env.example, then build/start there. Downloads are needed for the first source build. No separate host Node/PostgreSQL installation is required when using these containers.

Set POSTGRES_DB, POSTGRES_USER, a strong unique POSTGRES_PASSWORD, a strong random JWT_SECRET, the buyer's HOSPITAL_NAME/ADDRESS/PHONE, FRONTEND_PORT=8081, VITE_API_URL=/api and LIS_VERSION. Do not leave the development password or fallback JWT secret. URL-encode special characters when manually constructing DATABASE_URL; for the current interpolated Compose password, a securely generated long hexadecimal secret avoids URL-reserved characters.

Use only that hospital's authorized HMS connection/token if integrating it; otherwise leave those values empty. Never copy the seller's cloud, registry, database, HMS or administrator credentials. The first administrator is created in the initial setup screen; restrict network access to the installer until setup is complete. Create named accounts for staff, not a shared administrator login.

```
cd D:\lis
docker compose config --quiet
docker compose up -d --build
docker compose ps
```

Check frontend `/health` and `/api/health` at the chosen browser URL, complete initial setup, then test from a second PC. The Compose setup publishes only the frontend port. PostgreSQL `5432` and backend `4044` remain reachable only inside the private Compose network. Only browser traffic should cross the approved firewall boundary.

For a buyer who should not build/download dependencies, prepare and transfer prebuilt images instead. On your build machine, after successful tests and a versioned build:

```
docker compose build backend frontend
docker image save -o lis-1.1.0-images.tar `
  akash-lis-backend:1.1.0 akash-lis-frontend:1.1.0 postgres:16-alpine
```

Change all example version numbers together to the release actually built. Include the PostgreSQL image in the bundle. Match the server CPU/platform (these builds are not proof of ARM support). On the buyer server:

```
docker image load -i lis-1.1.0-images.tar
docker compose up -d --no-build --pull never
```

The buyer still needs a compatible runtime, buyer-specific root .env, Compose configuration and the database schema/seed folder referenced by its mount. For a source-free delivery, prepare a runtime-only copy of Compose by removing backend/frontend build sections while retaining the exact image tags and services. The current source Compose itself still includes build definitions. Transfer these runtime files with the image archive; do not ship personal .env files or hospital data. A packaged, tested one-click installer has not been produced yet.

For future offline updates: back up first, load the new archive, change LIS_VERSION to the new tag, then run docker compose up -d --no-build --pull never backend frontend. For private-registry delivery, give the hospital a narrowly scoped pull-only credential, never your publisher password; pull the release and recreate services. Updating the server updates what browser clients receive after reload. Clients do not rebuild anything.

7. Before commercial use

Agree ownership, support, licensing and data-retention terms; choose a supported server/runtime, plan recovery and monitor it. Docker Desktop has separate commercial licensing conditions; verify the hospital's eligibility rather than sharing your Docker account. Central Linux Docker Engine is an alternative for a professionally managed server, subject to deployment testing.

Verify HTTPS/private access, secret rotation, least-privilege database/network access, backups with a successful restore drill, release rollback, signed-image/update integrity and multi-user load. These are not proven merely because the build and functional tests pass. The existing commercial planning PDF explains the remaining rollout and licensing work.

Reference documentation

Docker Compose production guidance: https://docs.docker.com/compose/how-tos/production/

Compose up and container recreation: https://docs.docker.com/reference/cli/docker/compose/up/

Docker Desktop Windows installation and licensing notice:
https://docs.docker.com/desktop/setup/install/windows-install/

Project references: docs/doctor-access.md; docs/LIS_COMMERCIAL_DEPLOYMENT_AND_LICENSING_PLAN.pdf; backend/test/run-access-regression.mjs.